SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – MINI APPLICATION DEVELOPMENT (CHATBOT / SUMMARIZER / TRANSLATOR)

Course Code:	CSA6502	**Course Title:**	Generative AI and Large Scale Model
CO Assessed:	CO4 – Precision (BL3, BL4,BL5)	**Assessment:**	Assessment Tool 1-Mini Application Development (Chatbot / Summarizer / Translator)
Weightage:	20% of CIA	**Total Marks:**	2*50=100
CO4 Statement: Design and implement multimodal Generative AI applications integrating text, image, and speech models for engineering applications.(BL3, BL4,BL5)			
Student Name:	C. Praveen kumar	**Reg. No.:**	192472034
Section / Batch:	2024	**Date:**	20-08-2026

Code of Conduct

I, C. Praveen kumar / 192472034 (Name / Reg No), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: C. Praveen kumar

Name: C. Praveen kumar Reg.No: 192472034

Question 1: Tokenization

(a) Split "DISAGREEMENT" into 3 sub-word pieces

DIS + AGREE + MENT

Justification: This split separates the word into recognizable, reusable morphological units that a sub-word tokenizer (e.g., BPE or WordPiece) is likely to have learned from other words in its training data.

- "DIS" — a common negating prefix, also seen in "dislike," "disorder," "distrust."
- "AGREE" — the core root word, high-frequency on its own ("agree," "agreed," "agreeing").
- "MENT" — a common noun-forming suffix, also seen in "argument," "government," "enjoyment."

Because each piece independently appears in many other common words, the tokenizer's vocabulary is very likely to contain all three as learned merges, even if "DISAGREEMENT" itself never appeared as a whole token during training.

(b) Why sub-word tokenization helps

Sub-word tokenization lets the model represent any word — including rare or unseen ones — as a combination of smaller, frequently-seen pieces instead of needing every whole word in its vocabulary. Since the model has already learned the meaning and usage patterns of pieces like "DIS," "AGREE," and "MENT" from many other words, it can compose an understanding of "DISAGREEMENT" without ever having seen that exact word, which avoids the out-of-vocabulary problem that whole-word tokenizers face.

Question 2: Embedding

(a) Which pair has higher cosine similarity?

"dog" and "puppy" would have the HIGHER cosine similarity.

These two words are semantically related — they refer to the same animal (an adult dog vs. a young dog) and appear in very similar contexts (e.g., "walk the ___," "feed the ___," "pet the ___"). "dog" and "truck" are unrelated in meaning and typically appear in very different contexts, so their embedding vectors would point in more different directions, giving a lower cosine similarity. Embedding models learn similarity largely from how words are used in surrounding context, so words that co-occur with similar neighboring words end up closer together in the vector space.

(b) What an embedding represents vs. a one-hot vector

An embedding is a dense vector of real numbers (typically hundreds of dimensions) that encodes a word's meaning based on the contexts it tends to appear in — words with similar meanings or usage patterns end up close together in this vector space, and the geometric relationships between vectors (distance, direction) can even capture semantic relationships like analogies. A one-hot vector, in contrast, is a sparse vector as long as the entire vocabulary, with a single 1 marking that word's index and 0s everywhere else; it treats every word as equally different from every other word and carries no information about meaning or similarity at all.

Question 3: Self-Attention Worked Example

"The city council refused the protesters a permit because they feared violence."

(a) "They" refers to:

- [x] City council

(b) Why self-attention (not word order) is needed

A simple word-order rule (like "pronouns refer to the nearest preceding noun phrase") would incorrectly point "they" to "protesters," since that phrase is closer in the sentence. Resolving the reference correctly requires understanding the meaning of the whole sentence: it is the council's fear of violence that logically explains why they refused a permit, which is a semantic/contextual judgment rather than a positional one. Self-attention lets "they" directly attend to and weigh information from every other word in the sentence — regardless of distance — so the model can use cues like "refused" and "feared violence" to infer that "city council" is the more plausible antecedent.

[3.5 marks]

Question 4: Query, Key, Value

In self-attention, each input token is projected into three separate vectors — Query, Key, and Value — each capturing a different role in the attention computation:

- **Query (Q):** represents what the current token is "looking for" — the kind of information it needs from other tokens in order to update its own representation.
- **Key (K):** represents what each token "has to offer" — a label or signature that other tokens can match against when searching for relevant information.
- **Value (V):** represents the actual content or information that gets passed along once a token is deemed relevant — this is what actually gets aggregated into the output.

To combine them, the model computes a similarity score (typically a scaled dot product) between a token's Query vector and every other token's Key vector; these scores are passed through a softmax to produce attention weights that sum to 1. Each token's output representation is then a weighted sum of all the Value vectors, using those attention weights — so tokens whose Keys best match the Query contribute the most to the result.

[5 marks]

Practical Application: Educational Content Summarizer Project

Name: C. Praveen kumar Reg.No: 192472034

The concepts covered in Questions 1–4 (tokenization, embeddings, self-attention, and Query/Key/Value) are the exact mechanisms running inside the LLM backend of my own project, the Educational Content Summarizer. This section connects each worksheet answer to a concrete part of that project.

Project Overview

A Streamlit web application that converts lengthy educational material — pasted text, or an uploaded .txt / .pdf file — into a concise summary while preserving important concepts and domain terminology. It is powered by the Google Gemini API (selectable between gemini-2.5-flash and gemini-2.5-pro) through a single generate_content() call built with a structured prompt (app.py).

The user can configure summary length (Very short → Detailed), target audience level (Beginner → Graduate), and optionally request a bulleted list of key takeaways and a glossary of key terms with plain-language definitions. PDF text is extracted with pypdf before being sent to the model, and the final output can be downloaded as a Markdown file.

Pipeline

Content Input (paste or upload .txt/.pdf) → PDF Text Extraction (pypdf, if applicable) → Prompt Construction (length, audience level, glossary/bullet toggles built into the instruction) → Gemini Generation (model.generate_content()) → Formatted Output (Summary + Key Takeaways + Key Terms & Definitions in Markdown) → Download.

Mapping Worksheet Concepts to the Project

Q1 – Tokenization

Every piece of pasted or uploaded educational content — including subject-specific terminology such as "backpropagation," "cosine similarity," or institution-specific jargon — is broken into sub-word tokens by Gemini's tokenizer before the prompt is processed. This is exactly why the app can summarize material on topics or vocabulary it was never explicitly trained on: rare or compound terms are composed from familiar sub-word pieces, the same DIS + AGREE + MENT style decomposition from Q1, instead of failing on an out-of-vocabulary word.

Q2 – Embeddings

Unlike a retrieval system, the summarizer does not need its own embedding step — but internally, Gemini represents every input token as a dense embedding so it can recognize when different phrases in the source material express the same concept (much like "dog" and "puppy" from Q2(a) sitting close together in vector space). This is what lets the app write a fluent, reworded summary while still keeping the original technical terms intact, rather than treating every sentence as an unrelated bag of words the way a one-hot representation would.

Q3 – Self-Attention

Long study material often has pronouns and references that point back several sentences ("this technique," "the former approach," "they"). When Gemini generates the summary, self-attention lets each output token attend to every token of the source passage — regardless of distance — the same mechanism that resolved "they" to "city council" in Q3. This is why the generated summary correctly keeps track of what a later reference is pointing to, instead of losing the connection the way a purely positional or word-order rule would.

Q4 – Query, Key, Value

When the model generates the summary token by token, every token of the combined prompt (system instructions + the pasted/uploaded educational content) is projected into Query, Key, and Value vectors. Each output token's Query attends over the Keys of the source-material tokens, and the softmax-weighted Values pull the most relevant facts and terminology into the summary — this is the underlying mechanism that keeps the "important concepts and terminology" preserved rather than diluted, which is the core requirement of the project.

Engineering Notes Relevant to the Concepts Above

- Model choice is exposed to the user (gemini-2.5-flash for speed/cost, gemini-2.5-pro for higher-quality summaries) — tokenization and attention behavior differ slightly between the two, but the prompt-construction layer stays identical.
- Summary length and audience level are turned into explicit instructions inside the prompt, so the same source tokens are attended to differently depending on how much detail is requested.

- The Gemini API key is entered by the user at runtime and kept only in the session's memory — it is never written to disk.
- PDF uploads are handled with pypdf; scanned/image-only PDFs are out of scope since that would require OCR.

Application Screenshot

The screenshot below shows the app running locally, with a machine-learning lecture-notes excerpt pasted in and the Gemini-generated summary, key takeaways, and glossary produced on the right.

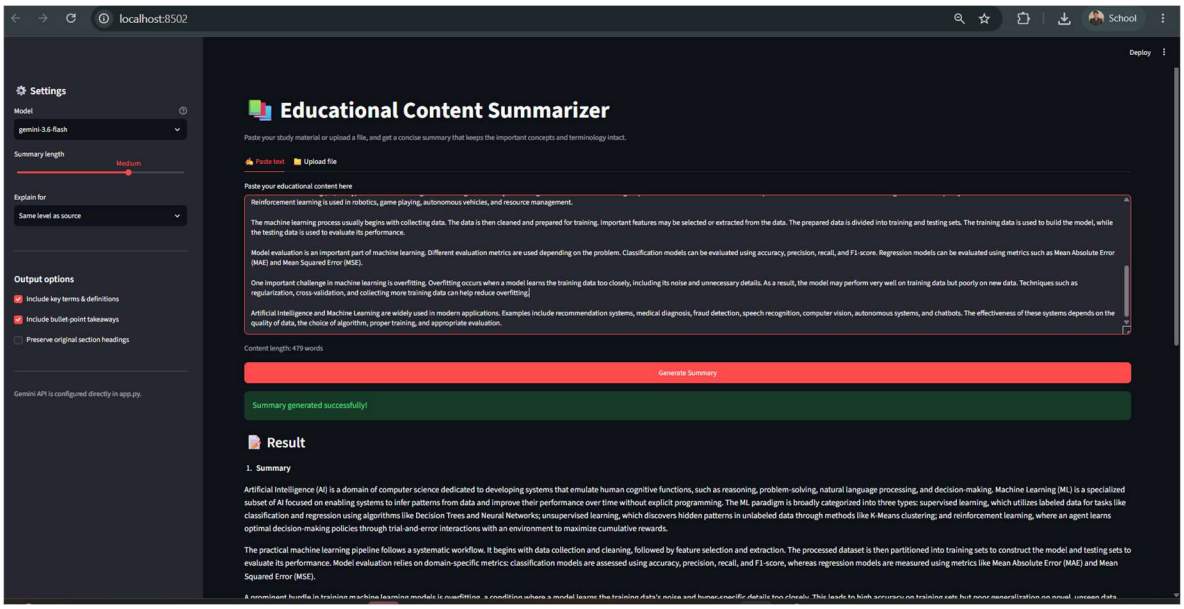

Fig. 1 — Educational Content Summarizer (Streamlit UI) generating a summary from pasted lecture notes.

Files referenced: app.py, requirements.txt, README.md.